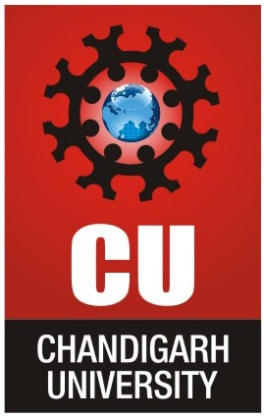
CHANDIGARH UNIVERSITY

Gharuan, Punjab



PROJECT REPORT

STUDENT MANAGEMENT SYSTEM USING C

Subject: C Programming

Submitted By

Priyam

UID: 26BCA10168

Bachelor of Computer Applications (BCA)

Submitted To

Khushi Agarwal Ma'am

Academic Session: 2026–2027

This is a student-prepared cover layout, not an official university template.

CERTIFICATE

This is to certify that the project report titled “Student Management System Using C” has been prepared by Priyam (UID: 26BCA10168), a Bachelor of Computer Applications student at Chandigarh University, for the subject C Programming during the academic session 2026–2027.

The project demonstrates basic C programming concepts through a menu-driven application for maintaining student records, searching by UID, updating marks and attendance, and deleting records. The program and report are submitted for academic learning and evaluation.

This page is provided as a report format. It does not contain an actual faculty signature or approval. The teacher may sign or update it if required by the university.

Student Signature: _____

Faculty Signature: _____

Date: _____

ACKNOWLEDGEMENT

I would like to thank my C Programming teacher, Khushi Agarwal Ma'am, for giving me the opportunity to prepare this project on Student Management System Using C.

This project helped me practise variables, conditions, loops, functions, arrays, structures, pointers, strings, and file handling. I learned how a menu makes a program easier to use and how files keep student records available after the program is closed.

I am also thankful to Chandigarh University for providing an academic environment where I can learn programming through practical work. I have tried to keep the code simple so that each function can be understood and explained during viva.

The project is a small educational model. It does not represent a deployed university student information system or official academic database.

TABLE OF CONTENTS

S. No.	Section	Page
1	Introduction to Student Management System	5
2	Problem Statement and Objectives	5
3	Hardware and Software Requirements	6
4	C Programming Concepts Used	6
5	System Design and Working	7
6	Algorithm and Flowchart	8
7	Source Code Explanation	9
8	Sample Output	10
9	Advantages, Limitations and Future Scope	11
10	Conclusion and References	12

Note: Page numbers are planned for this report layout. If you make large edits in Word, update the page numbers to match the final pagination.

1. INTRODUCTION

Educational institutions maintain details such as student UID, name, course, semester, email, marks, and attendance. When these records are maintained manually, searching for a student or updating information can take time and may lead to errors.

The Student Management System in this project is a console-based application written in C. It allows the user to add a student, view all records, search by UID, update marks, update attendance, and delete a student record. Student data is saved in a file so records remain available after the program is closed.

This project is designed for learning the fundamentals of C programming. It is not a complete institutional student information system because it does not include user authentication, online access, or integration with a university database.

2. PROBLEM STATEMENT

The main problem addressed is how to keep a small set of student records organised using a basic C program. Manual record keeping can lead to repeated entries, slow searches, and confusion when student details, marks, or attendance need to be updated.

OBJECTIVES

- Create a menu that is simple to understand.
- Store each student using a UID, name, course, semester, email, marks, and attendance percentage.
- Prevent duplicate student UIDs and validate marks and attendance values.
- Allow the user to search, update, and delete student records.
- Save records using file handling so they persist between program runs.
- Practise functions, structures, arrays, pointers, loops, conditions, strings, and file handling.

3. HARDWARE AND SOFTWARE REQUIREMENTS

Type	Requirement
Hardware	Desktop or laptop computer
Processor	Any modern entry-level processor
Memory	2 GB RAM or more is sufficient for this small program
Operating system	Windows, Linux, or macOS
Editor	Visual Studio Code or another text editor
Compiler	GCC or another standard C compiler
Storage	A few MB for source, executable, and data file

4. C PROGRAMMING CONCEPTS USED

Concept	How it is used
Variables and data types	Integers store UIDs and semester; floating-point values store marks and attendance; character arrays store text.
if-else	Checks invalid UIDs, missing students, and out-of-range values.
switch-case	Selects an action from the main menu.
Loops	The main loop repeatedly shows the menu; loops also read and shift records.
Functions	Separate functions handle add, display, search, update, delete, loading, and saving.
Arrays	An array stores multiple Student structures in memory.
Structures	Student groups UID, name, course, semester, email, marks, and attendance.
Pointers	Pointers pass the record count to functions so it can be updated.
String handling	fgets and string functions read and check student names, courses, and email addresses.

File handling	fopen, fread, fwrite, fclose, remove, and rename save and load records.

5. SYSTEM DESIGN AND WORKING

5.1 Data structure

Each student is represented by a structure named Student. It contains an integer UID, character arrays for name, course, and email, an integer semester, and floating-point values for marks and attendance percentage.

5.2 Menu options

Option	Action	Main check
1	Add New Student	UID must be positive and unique; marks and attendance must be within 0–100.
2	Display All Students	Shows a message if there are no records.
3	Search Student by UID	Finds the matching UID or reports not found.
4	Update Student Marks	Accepts marks only within the valid range.
5	Update Attendance	Accepts attendance only from 0 to 100 percent.
6	Delete Student Record	Deletes the selected student record.
7	Exit	Ends the program.

5.3 File storage

At startup, loadStudents reads records from students.dat into an array. After a change, saveStudents writes the current records to a temporary file and renames it to students.dat. The data file is created in the program's current working directory.

5.4 Error handling

The program checks numeric input, positive UIDs, duplicate UIDs, valid marks and attendance percentages, missing records, and file errors. It is intended for one user at a time.

6. ALGORITHM AND FLOWCHART

6.1 Main algorithm

1. Start the program.

2. Load saved student records from students.dat.

3. Display the main menu.

4. Read the user's choice and validate it.

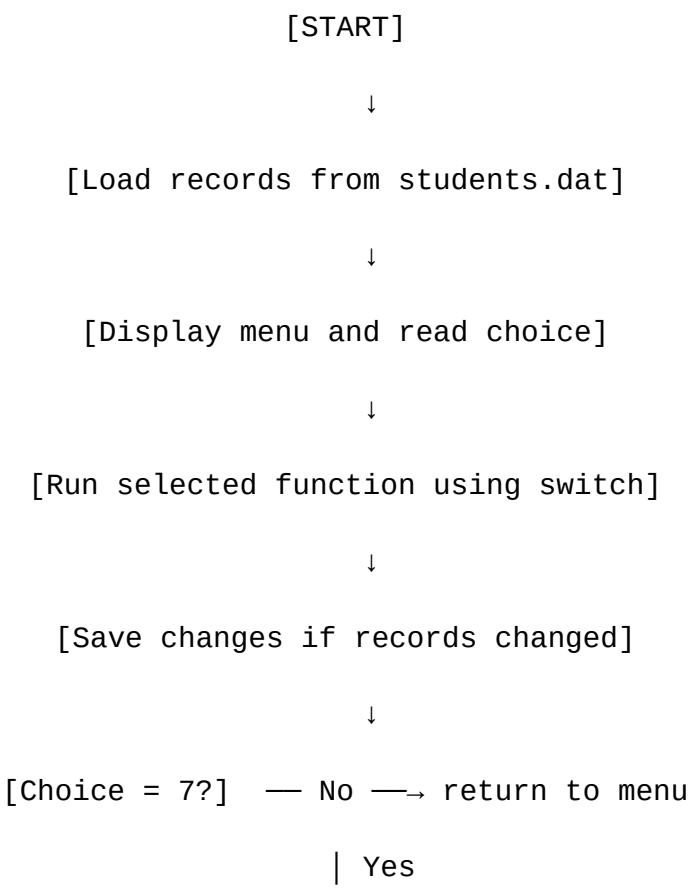
5. If a record changes, save the updated array to the data file.

6. If a record changes, save the updated array to the data file.

7. Repeat the menu until the user chooses Exit.

8. End the program.

6.2 Flowchart



↓

[END]

The flowchart is shown as editable text to keep it clear in Word. It describes the control flow of the actual source code.

7. SOURCE CODE EXPLANATION

7.1 Header files and constants

stdio.h provides input and output functions. stdlib.h provides general utilities, while string.h is used for string handling. Constants define the data file names and maximum field sizes.

7.2 Structure and functions

The Student structure keeps related fields together. loadStudents reads saved structures into the array. saveStudents writes the array to disk. findStudentIndex searches the array and returns the matching position, or -1 if the UID is absent.

7.3 Adding and displaying

addStudent asks for the UID, name, course, semester, email, marks, and attendance percentage. It checks that the UID is unique and values are within valid ranges. The new structure is added to the array, and the records are saved. displayStudents prints all records in columns.

7.4 Issue, return, and delete

updateMarks changes a student's marks after validating the value. updateAttendance changes the attendance percentage within the range 0 to 100. deleteStudent shifts the later array entries left to fill the removed position.

7.5 Main function

main loads existing records and runs a menu loop. The switch statement calls the function for the selected option. Choosing 7 prints a message and exits.

7.6 Important note

The source code is provided separately in student.c. Keep it with this report and the README. The report's explanation matches the supplied source code.

8. SAMPLE OUTPUT

The following are console transcripts from the program's expected interaction pattern. They are text examples, not photographs of a terminal window. You can compile and run student.c to capture your own screenshots for final submission.

8.1 Main menu

```
========== STUDENT MANAGEMENT SYSTEM ==========

1. Add New Student

2. Display All Students

3. Search Student by UID

4. Update Student Marks

5. Update Attendance

6. Delete Student Record

7. Exit

Enter your choice (1-7):
```

8.2 Add and display a student

```
Enter your choice (1-7): 1

Enter student UID: 101

Enter student name: Aarav Sharma

Enter course: BCA

Enter semester: 1

Enter email: aarav@example.com


UID      NAME                COURSE   SEMESTER   MARKS   ATTENDANCE

101      Aarav Sharma        BCA      1          85.5    92.0%
```

8.3 Search, issue, return, and delete

Search: enter UID 101 → “Student found” followed by the student details.

Update marks: enter UID 101 and marks 88 → “Marks updated successfully”.

Update attendance: enter UID 101 and attendance 95 → “Attendance updated successfully”.

Delete: enter UID 101 → “Student record deleted successfully”.

These examples use sample student information only. They do not represent actual university records.

9. ADVANTAGES AND LIMITATIONS

9.1 Advantages

- The menu is straightforward for a beginner to use.
- Student information is grouped neatly in a structure.
- Searching by UID is simple for a small record list.
- The program validates marks and attendance before updating records.
- Records are saved to a file and remain after the program exits.
- The code is divided into functions, making it easier to read and test.

9.2 Limitations

- There is no separate login system or role-based access for students and administrators.
- The project does not connect to a live university database or provide online student/parent access.
- The program is designed for one user at a time.
- The binary data file should be used with the matching program structure and compiler environment.
- The fixed-size array limits the number of records to 1,000.

10. FUTURE SCOPE

The project can be improved by adding login authentication, course-wise reports, GPA calculation, fee records, timetable management, and a graphical interface. A database, backup, and export options could be added in a later version.

11. CONCLUSION

The Student Management System demonstrates how basic C programming concepts can solve a small record-keeping problem. Structures store student details, functions separate tasks, a menu allows the user to select operations, and file handling keeps records between runs.

While making this project, I practised validating input and checking conditions before changing records. The project is suitable as a BCA learning exercise. It is a simple console program and would need more features and security controls before use in a real institution.

12. REFERENCES

- The C Programming Language, Brian W. Kernighan and Dennis M. Ritchie, Prentice Hall.
- GCC online documentation: https://gcc.gnu.org/onlinedocs/
- Microsoft Learn, C/C++ development with Visual Studio Code: https://code.visualstudio.com/docs/languages/cpp
- Class notes and learning material for the C Programming subject.

The project code and sample records were prepared for academic practice. No actual student survey, interview, or institutional database is claimed.